

# Aula 28 - Segurança em APIs Reativas com Spring Security WebFlux

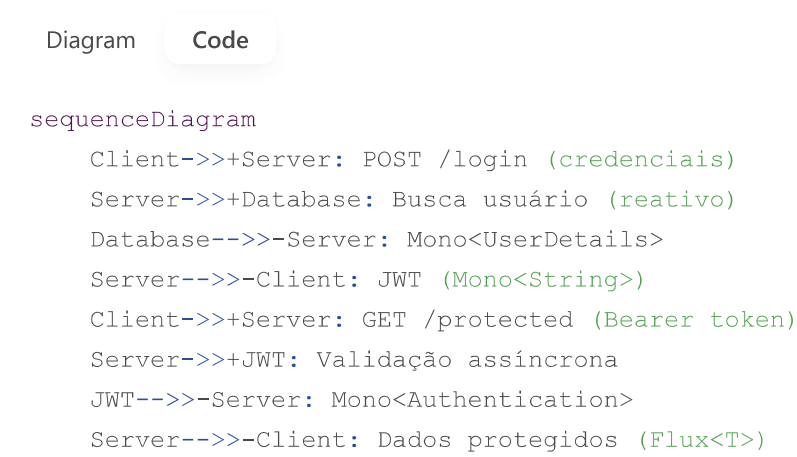
Duração: 4 horas (1h teórica + 3h prática)

## Parte Teórica (1 hora)

### 1. Diferenças para o Spring MVC (20 min)

| Componente         | Spring MVC (Blocking) | Spring WebFlux (Reactive)         |
|--------------------|-----------------------|-----------------------------------|
| Security Filter    | FilterChain           | WebFilter                         |
| UserDetailsService | UserDetailsService    | ReactiveUserDetailsService        |
| Password Encoder   | BCryptPasswordEncoder | PasswordEncoder (mesma interface) |
| Autenticação       | SecurityContextHolder | ReactiveSecurityContextHolder     |

### 2. Fluxo de Autenticação Reativa (20 min)



### 3. Anotações e Configurações (20 min)

- `@EnableReactiveMethodSecurity` : Habilita segurança em métodos reativos.
- `ServerHttpSecurity` : Configuração de rotas e políticas (equivalente reativo do `HttpSecurity`).
- `AuthenticationWebFilter` : Filtro customizado para JWT.

# Parte Prática (3 horas)

## Atividade: Autenticação JWT em WebFlux

### Passo 1: Configuração Inicial (1h)

#### 1. Dependências ( pom.xml ):

```
xml

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
```

#### 2. Classe de Configuração ( SecurityConfig.java ):

```
java

@EnableWebFluxSecurity
@EnableReactiveMethodSecurity
public class SecurityConfig {
    @Bean
    public SecurityWebFilterChain securityWebFilterChain(ServerHttpSecurity http) {
        return http
            .csrf().disable()
            .authorizeExchange()
                .pathMatchers("/auth/**").permitAll()
                .anyExchange().authenticated()
            .and()
            .addFilterAt(jwtAuthFilter(), SecurityWebFiltersOrder.AUTHENTICATION)
            .build();
    }
}
```

### Passo 2: Implementação do JWT (1h)

#### 1. Filtro de Autenticação ( JwtAuthFilter.java ):

```
java

public class JwtAuthFilter implements WebFilter {
    @Override
```

```

        public Mono<Void> filter(ServerWebExchange exchange, WebFilterChain chain) {
            String token = extractToken(exchange.getRequest());
            if (token == null) return chain.filter(exchange);

            return Mono.just(token)
                .flatMap(jwtUtil::validateToken)
                .flatMap(auth -> chain.filter(exchange)
                    .contextWrite(ReactiveSecurityContextHolder.withAuthentication(auth)))
                .then();
        }
    }
}

```

## 2. Serviço de Autenticação ( AuthService.java ):

```

java

@Service
@RequiredArgsConstructor
public class AuthService {
    private final ReactiveUserDetailsService userDetailsService;
    private final PasswordEncoder encoder;

    public Mono<Authentication> authenticate(String username, String password) {
        return userDetailsService.findByUsername(username)
            .filter(user -> encoder.matches(password, user.getPassword()))
            .map(user -> new UsernamePasswordAuthenticationToken(
                user, null, user.getAuthorities()
            ));
    }
}

```

## Passo 3: Endpoints de Login (1h)

### 1. Controller ( AuthController.java ):

```

java

@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class AuthController {
    private final AuthService authService;
    private final JwtUtil jwtUtil;

    @PostMapping("/login")
    public Mono<String> login(@RequestBody AuthRequest request) {
        return authService.authenticate(request.username(), request.password())
            .flatMap(jwtUtil::generateToken);
    }
}

```

```
}  
}
```

## 2. DTOs:

java

```
public record AuthRequest(String username, String password) {}
```

## Tarefa para Casa

1. **Refresh Token:** Implementar renovação de token via endpoint `/auth/refresh`.
2. **Controle de Acesso:** Criar um método seguro `@PreAuthorize("hasRole('ADMIN')")`.

## Próxima Aula

- **Tópico:** WebSockets com WebFlux.
- **Atividade prática:** Chat em tempo real.

## Resultado Esperado

- API reativa segura com:
  - Autenticação JWT não-blocking.
  - Rotas protegidas por roles.
  - Integração com R2DBC (usuários no banco).

plaintext

- ✅ Login reativo funcionando
- ✅ Rotas protegidas com JWT
- ✅ Pronto para WebSockets

<https://spring.io/images/spring-reactive-security-9c1b5e5c5a5e8e5e5e5e5e5e5e5e5e5e.png> (Fluxo de segurança reativa) 🚀